INFORME DEL TRABAJO

I. PORTADA

Tema:	Prueba Practica Segundo Parcial
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumno participante:	Camacho Monta Josue Jampier Chalco Tasna Kenneth Mateo Cunalata Mendoza Damian Alexander Silva Camuendo Luis Alexander Tacuri Santillan Monica Sara Tisalema Guashco Darwin Joel ……..
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza, Mg.

INFORME DEL TRABAJO

2.1 Resultados obtenidos

Resultados obtenidos de la actividad Prueba Practica Segundo Parcial

1. Ejercicio

Desarrollar una aplicación de consola en Java que permita auditar la estructura académica de la Universidad Técnica de Ambato y administrar las matrículas de sus estudiantes.

La jerarquía académica se representa mediante un árbol general, mientras que los estudiantes se almacenan en un árbol binario de búsqueda —ABB— ordenado por matrícula.

El sistema debe validar los datos, evitar identificadores duplicados, permitir consultas recursivas, realizar eliminaciones seguras y generar un reporte integrado. También debe transformar el árbol general mediante la técnica hijo izquierdo–hermano derecho y comparar búsquedas en dos ABB de diferente forma.

2. Solución propuesta

La solución se organiza en cuatro paquetes principales: modelo, estructuras, servicio y app.

ArbolGeneral administra las unidades académicas y ArbolEstudiantes gestiona las matrículas. La clase Auditoria consulta ambas estructuras y prepara un reporte; AnalisisBusqueda compara el costo de buscar en un árbol en cadena y otro construido mediante inserciones por medianas.

Main permite utilizar estas funcionalidades desde un menú. Las pruebas se encuentran separadas en test/pruebas.

En esta implementación, los estudiantes pertenecen a un ABB global; el código no los vincula individualmente con los paralelos del árbol general.

3. ¿Qué son el árbol general y el ABB?

El árbol general permite que cada nodo tenga varios hijos. Representa relaciones jerárquicas entre universidad, facultades, carreras y paralelos.

El ABB admite hasta dos hijos por nodo. Todas las matrículas del subárbol izquierdo deben ser menores que la matrícula del nodo, y todas las del derecho deben ser mayores.

La jerarquía inicial es:

Unidad	Padre	Tipo
UTA	Ninguno	UNIVERSIDAD
FISEI	UTA	FACULTAD
FCHE	UTA	FACULTAD
FCS	UTA	FACULTAD
SOFTWARE	FISEI	CARRERA
TI	FISEI	CARRERA
3A	SOFTWARE	PARALELO
3B	SOFTWARE	PARALELO
4A	TI	PARALELO

Esta estructura tiene 9 unidades, 5 hojas y altura 3, medida en aristas. Las hojas son FCHE, FCS, 3A, 3B y 4A.

Las matrículas se insertan en este orden:
50, 30, 70, 20, 40, 60, 80, 35, 45, 65, 75, 85

4. Relación entre requisitos y código

Requisito	Implementación
Validar tipos de padre e hijo	TipoUnidad.puedeTenerHijo()
Evitar códigos académicos repetidos	ArbolGeneral.agregar()
Buscar unidades	buscar() y existe()
Consultar nivel y grado	nivel() y grado()
Calcular métricas del árbol general	altura(), contarNodos() y contarHojas()
Imprimir la jerarquía	imprimirJerarquia()
Eliminar únicamente hojas no raíz	ArbolGeneral.eliminar()
Convertir a hijo izquierdo–hermano derecho	transformarABinario()
Validar matrícula, nombre y promedio	Constructor y validaciones de Estudiante
Insertar y buscar estudiantes	ArbolEstudiantes.insertar() y buscar()
Obtener recorridos	recorrer() y recorrerMatriculas()
Modificar sin cambiar matrícula	modificar()
Resolver los tres casos de eliminación	ArbolEstudiantes.eliminar()
Obtener mínimo, máximo y promedio	minimo(), maximo() y promedioGeneral()
Verificar el orden global del ABB	esABBValido()
Comparar búsquedas	AnalisisBusqueda.comparar()
Integrar resultados	Auditoria.generarReporte()
Ejecutar pruebas	EjecutarPruebas.main()

5. Paso 1: Organizar los paquetes e importar las librerías

Paquete	Archivos
modelo	Estudiante.java, UnidadAcademica.java, TipoUnidad.java
estructuras	ArbolGeneral.java, ArbolEstudiantes.java, NodoUnidad.java, NodoEstudiante.java, NodoBinarioUnidad.java, TipoRecorrido.java
servicio	Auditoria.java, AnalisisBusqueda.java
app	Main.java, DatosPrueba.java

Las herramientas principales son:

- **Scanner:** lectura de entradas.
- **ArrayList y List:** hijos del árbol general y resultados de recorridos.
- **StringBuilder:** construcción de reportes y representaciones.
- **Arrays:** copia y ordenamiento del arreglo utilizado en el análisis comparativo.

Arrays.sort() prepara los datos del experimento de búsqueda; los recorridos del ABB se obtienen recorriendo sus nodos, sin ordenar posteriormente sus resultados.

6. Paso 2: Crear las enumeraciones y las clases del modelo
TipoUnidad establece cuatro niveles:

```
UNIVERSIDAD,
FACULTAD,
CARRERA,
PARALELO;
```

La relación permitida se comprueba mediante:

```
public boolean puedeTenerHijo(TipoUnidad tipoHijo) {
   return tipoHijo != null
         && tipoHijo.ordinal() == this.ordinal() + 1;
}
```

Así, una universidad admite facultades; una facultad, carreras; y una carrera, paralelos. La validación depende del orden de las constantes de la enumeración.

UnidadAcademica almacena código, nombre y tipo. El código y el tipo son final. Los códigos se normalizan quitando espacios exteriores y convirtiéndolos a mayúsculas.

Estudiante almacena matrícula, nombre y promedio:

```
private final int matricula;
private String nombre;
private double promedio;
```

La matrícula debe ser positiva y no tiene setter. El nombre no puede estar vacío y el promedio debe encontrarse entre 0 y 10; también se rechaza NaN.

TipoRecorrido define PREORDEN, INORDEN y POSORDEN.

7. Paso 3: Crear los nodos y conectar las estructuras
Se utilizan tres tipos de nodos:

Nodo	Dato almacenado	Enlaces
NodoUnidad	UnidadAcademica	Lista de hijos
NodoEstudiante	Estudiante	Izquierdo y derecho
NodoBinarioUnidad	UnidadAcademica	Primer hijo y siguiente hermano

NodoUnidad inicializa su lista de hijos:

```
this.hijos = new ArrayList<>();
```

Su método getHijos() devuelve una vista de solo lectura:

```
return java.util.Collections.unmodifiableList(hijos);
```

Esto impide modificar directamente la lista devuelta. Para mantener las validaciones globales, las operaciones normales deben realizarse mediante ArbolGeneral.

Los nodos binarios académicos representan la conversión de la jerarquía. No constituyen un ABB, porque sus enlaces representan parentesco y no comparaciones de claves.

8. Paso 4: Crear ArbolBiblioteca y la raíz
ArbolGeneral comienza vacío: su referencia raiz vale null.

La raíz se crea con crearRaiz(), que rechaza una segunda raíz, un dato nulo o una unidad cuyo tipo no sea UNIVERSIDAD.

```
public boolean crearRaiz(UnidadAcademica unidad) {
   if (raiz != null || unidad == null
         || unidad.getTipo() != TipoUnidad.UNIVERSIDAD) {
      return false;
   }
   raiz = new NodoUnidad(unidad);
   return true;
}
```

Para agregar una unidad, agregar() comprueba que exista raíz, que el código no esté registrado y que el padre exista. Después crea un nodo nuevo y valida la relación jerárquica.

La búsqueda recorre el nodo actual y, si no coincide, sus hijos recursivamente. Como los códigos no siguen un orden de búsqueda, puede ser necesario visitar todas las unidades.

La eliminación protege la raíz y rechaza unidades con descendientes. Por ejemplo, eliminar FISEI se rechaza; eliminar 4A se permite porque es una hoja.

9. Paso 5: Insertar, buscar, modificar y eliminar libros
La inserción comprueba primero si la matrícula existe. Si está repetida, devuelve false; en caso contrario, desciende recursivamente hasta una posición vacía:

```
private NodoEstudiante insertar(
      NodoEstudiante nodo, Estudiante estudiante) {

   if (nodo == null) {
      return new NodoEstudiante(estudiante);
   }

   if (estudiante.getMatricula() < nodo.getMatricula()) {
      nodo.setIzquierdo(
            insertar(nodo.getIzquierdo(), estudiante));
   } else {
      nodo.setDerecho(
            insertar(nodo.getDerecho(), estudiante));
   }

   return nodo;
}
```

La búsqueda elige una sola rama según la matrícula. buscar() reinicia y actualiza un contador que registra un nodo visitado por comparación contabilizada.

La modificación valida el nuevo nombre y promedio antes de aplicar cambios. La matrícula permanece igual.

La eliminación contempla:

Caso	Acción
Hoja	Devuelve null
Un hijo	Devuelve el hijo existente
Dos hijos	Sustituye el dato por el sucesor inorden y elimina su posición anterior

El fragmento central es:

```
if (nodo.getIzquierdo() == null) {
   return nodo.getDerecho();
}
if (nodo.getDerecho() == null) {
   return nodo.getIzquierdo();
}

NodoEstudiante sucesor = minimoNodo(nodo.getDerecho());
nodo.setDato(sucesor.getDato());
nodo.setDerecho(
      eliminar(nodo.getDerecho(), sucesor.getMatricula()));
```

Se reemplaza el objeto almacenado en el nodo; no se modifica la matrícula final del objeto original.

10. Paso 6: Implementar consultas recursivas y análisis
Las métricas se calculan visitando los nodos:

Consulta	Lógica
Total de nodos	Uno por el nodo actual más los resultados de sus hijos
Hojas	Cuenta los nodos que no tienen hijos
Altura	Uno más la mayor altura de los hijos
Nivel	Acumula la distancia desde la raíz
Grado	Cantidad de hijos directos
Mínimo del ABB	Desciende por enlaces izquierdos
Máximo del ABB	Desciende por enlaces derechos
Promedio general	Suma de promedios dividida entre estudiantes
Validez del ABB	Verifica límites estrictos heredados de los antecesores

La altura del ABB se implementa así:

```
private int altura(NodoEstudiante nodo) {
   if (nodo == null) {
      return -1;
   }
   return 1 + Math.max(
         altura(nodo.getIzquierdo()),
         altura(nodo.getDerecho()));
}
```

Por tanto, un árbol vacío tiene altura -1 y uno con un solo nodo tiene altura 0.

La conversión académica utiliza:

```
nodo.setIzquierdo(convertir(actual.getHijos(), 0));
nodo.setDerecho(convertir(hermanos, indice + 1));
```

El enlace izquierdo conserva el primer hijo; el derecho conserva el siguiente hermano. Se crean nuevos nodos binarios, sin cambiar los enlaces del árbol general.

Para analizar las búsquedas se construyen dos ABB con las mismas doce matrículas:

Construcción	Altura esperada	Buscar 85	Buscar 90
Inserciones ascendentes	11	12 nodos visitados	12 nodos visitados
Inserciones por medianas	3	4 nodos visitados	4 nodos visitados

El segundo árbol no se autoequilibra: su forma inicial depende del orden de inserción.

11. Paso 7: Construir datos iniciales menú recorridos y pruebas
DatosPrueba.cargarJerarquia() crea las nueve unidades. cargarEstudiantes() inserta las doce matrículas con sus nombres y promedios.

Main carga los datos al iniciar y ofrece:

Opciones	Funcionalidad
1–5	Consultas, inserción, eliminación y transformación del árbol general
6–10	Operaciones sobre estudiantes y recorridos
11	Auditoría integrada
12	Comparación de búsquedas
13	Escenario completo del enunciado
14	Reiniciar los datos
0	Salir

Las opciones 13 y 14 recargan los datos, por lo que descartan las modificaciones anteriores de la sesión.

Los recorridos iniciales esperados son:

Recorrido	Secuencia
Preorden	50, 30, 20, 40, 35, 45, 70, 60, 65, 80, 75, 85
Inorden	20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85
Posorden	20, 35, 45, 40, 30, 65, 60, 75, 85, 80, 70, 50

La visita ocurre antes de los hijos en preorden, entre ambos subárboles en inorden y después de ellos en posorden.

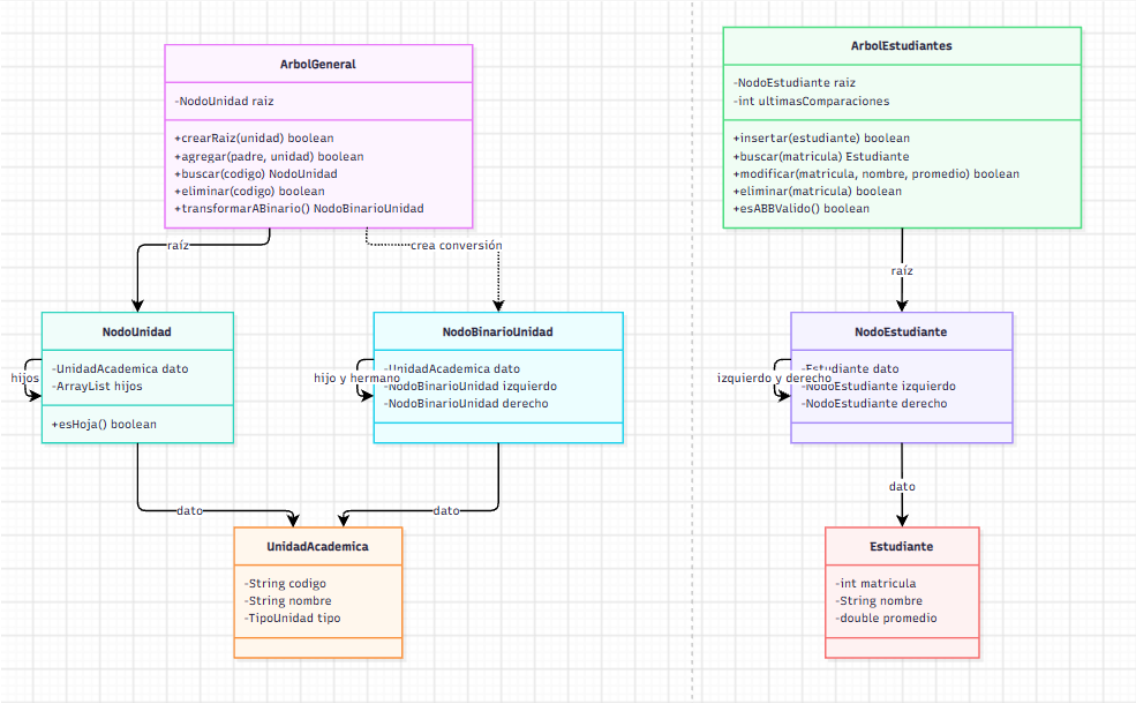
Las pruebas utilizan Verificador, que comprueba condiciones y genera AssertionError cuando una expectativa no se cumple. Su ejecución se presenta al final.

12. Tipos de datos utilizados

Tipo	Uso
int	Matrículas, opciones, alturas, niveles y contadores
double	Promedios y sumas de calificaciones
long	Límites de validación del ABB
boolean	Resultados de operaciones y validaciones
String	Códigos, nombres, mensajes y reportes
int[]	Matrículas utilizadas en la carga y el análisis
String[]	Nombres iniciales
double[]	Promedios iniciales

ArrayList<NodoUnidad>	Hijos del árbol general
ArrayList<Estudiante>	Resultados de recorridos
ArrayList<Integer>	Matrículas de los recorridos
Referencias a nodos	Raíces, hijos y hermanos
Scanner	Lectura de entradas
StringBuilder	Construcción de textos

13. Diagrama de clases
El siguiente diagrama resume las relaciones estructurales principales:



Main utiliza los árboles y los servicios. Auditoria consulta ambas estructuras, mientras que AnalisisBusqueda construye y consulta los ABB del experimento.

14. Especificación de la estructura
El árbol general mantiene una raíz de tipo UNIVERSIDAD. Cada relación debe respetar la secuencia universidad–facultad–carrera–paralelo.

El ABB mantiene matrículas únicas y la propiedad de orden en todos sus subárboles. La modificación de datos no cambia la clave.

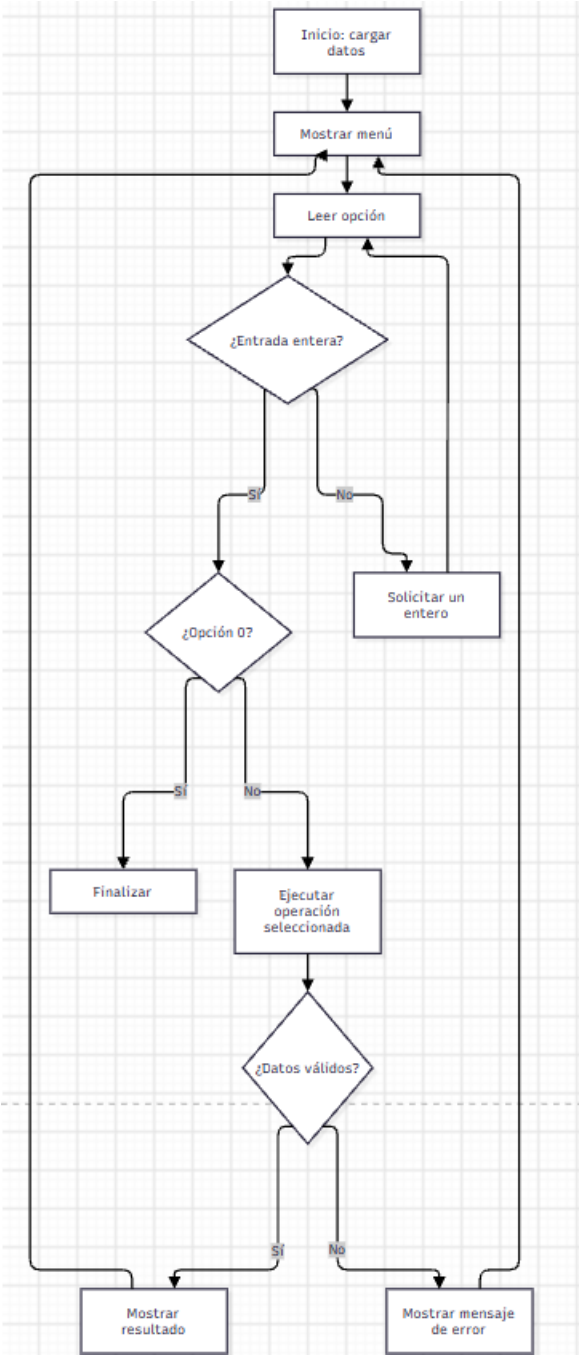
Operaciones:

Operación	Resultado o condición
crearRaiz(unidad)	false si ya existe raíz, el dato es nulo o el tipo es incorrecto
agregar(padre, unidad)	false ante duplicado, padre inexistente o relación inválida
buscar(codigo)	Nodo encontrado o null
nivel(codigo)	Profundidad; -1 si no existe
grado(codigo)	Hijos directos; -1 si no existe
altura()	Altura en aristas; vacío -1, nodo único 0
eliminar(codigo)	Rechaza raíz, unidad no hoja o inexistente
transformarABinario()	Nueva representación binaria; null si está vacío
insertar(estudiante)	Rechaza duplicados; dato nulo genera excepción
buscar(matricula)	Estudiante o null; registra nodos visitados

modificar(...)	Conserva matrícula; valida antes de modificar
eliminar(matricula)	Reconecta el árbol; false si no existe
minimo() y maximo()	Estudiante correspondiente o null
promedioGeneral()	Promedio aritmético; 0.0 si está vacío
esABBValido()	Comprueba límites estrictos; vacío válido

Alcance de las garantías: las validaciones globales de unidades se aplican en ArbolGeneral.agregar(). Los métodos públicos de NodoUnidad permiten saltarse parte de esas comprobaciones; por ejemplo, agregar directamente un nodo no comprueba códigos únicos en todo el árbol. Los setters de los nodos del ABB también permiten construir enlaces inválidos, como se hace deliberadamente en una prueba.

15. Diagrama de flujo del programa principal



Las entradas numéricas incorrectas se solicitan nuevamente. Las validaciones que lanzan IllegalArgumentException se muestran como mensajes de error. Una opción desconocida genera un aviso y el menú continúa.

El costo de las operaciones es:

Operación	Complejidad temporal
Buscar una unidad	O(N)
Métricas y conversión del árbol general	O(N)
Buscar, insertar o eliminar en el ABB	O(h + 1)
Recorrer, contar, calcular promedio o validar el ABB	O(n)
Mínimo y máximo del ABB	O(h + 1)

N representa unidades académicas, n estudiantes y h la altura del ABB. Habitualmente la búsqueda se expresa como O(h); h + 1 contempla también la visita a la raíz.

16. Prueba de escritorio
Los siguientes resultados se deducen de las operaciones implementadas:

Caso	Seguimiento	Resultado esperado
Árbol vacío	No existe raíz	Altura -1, cero nodos y hojas
Solo raíz UTA	No tiene hijos	Altura 0, una hoja
Jerarquía inicial	Se cargan las nueve unidades	9 nodos, 5 hojas, altura 3
Nivel de 3B	UTA → FISEI → SOFTWARE → 3B	Nivel 3
Grado de FISEI	Tiene SOFTWARE y TI	Grado 2
Duplicar matrícula 40	Se encuentra la matrícula existente	Rechazo; permanecen 12 estudiantes
Modificar matrícula 35	Nombre nuevo y promedio 9.6	Clave y posición conservadas
Eliminar 20	Es una hoja	El izquierdo de 30 pasa a null
Eliminar 60	Tiene como único hijo a 65	65 ocupa su posición
Eliminar 70	Tiene dos hijos; sucesor 75	75 ocupa su posición anterior
Eliminar FISEI	Tiene descendientes	Rechazo; permanecen 9 unidades
Convertir UTA	Primer hijo FISEI; siguientes FCHE y FCS	Se conserva el orden de hermanos

Después de eliminar 20, 60 y 70:

Inorden:
30, 35, 40, 45, 50, 65, 75, 80, 85

Preorden:
50, 30, 40, 35, 45, 75, 65, 80, 85

Quedan 9 estudiantes.

El promedio inicial se calcula con:

$$\frac{97}{12} = 8.0833\ldots$$

En el escenario de la opción 13 se modifica el promedio de la matrícula 35 de 6.5 a 9.6. Después se eliminan los estudiantes 20, 60 y 70, cuyos promedios son 6.8, 7.5 y 9.1:

$$\frac{97 - 6.5 + 9.6 - 6.8 - 7.5 - 9.1}{9} = \frac{76.7}{9} = 8.5222\ldots$$

La auditoría debe mostrar 8.52, con matrícula menor 30, mayor 85 e invariante VÁLIDA.

17. Reflexión personal

17.1. ¿Por qué no es correcto modificar la matrícula de un nodo sin reposicionarlo en un ABB?

Porque la matrícula determina su posición. Cambiarla puede dejar un valor mayor en el subárbol izquierdo o uno menor en el derecho, rompiendo el orden y haciendo que la búsqueda no encuentre datos existentes. En nuestro código es final y no tiene setter. Para cambiar una matrícula habría que eliminar el registro anterior e insertar uno nuevo, validando duplicados.

17.2. Trace la eliminación de un nodo con dos hijos y explique por qué el sucesor inorden mantiene el orden del ABB.

Después de eliminar 20 y 60, el nodo 70 tiene a 65 a la izquierda y a 80 a la derecha. Para eliminarlo buscamos el menor de su subárbol derecho: desde 80 bajamos a 75.

El dato de 75 reemplaza al de 70 y se elimina el 75 de su ubicación anterior. El orden se conserva porque 75 es mayor que todos los valores del subárbol izquierdo y menor que los restantes del derecho.

17.3. Si ambos árboles tienen n nodos, ¿por qué la búsqueda en uno puede ser O(n) y en otro O(log n)?

Porque el costo depende de la altura. Las inserciones ascendentes forman una cadena de altura n − 1; una búsqueda puede visitar los n nodos. Si las inserciones producen un árbol equilibrado, la altura crece de forma logarítmica.

En nuestro experimento, buscar 85 o 90 visita 12 nodos en la cadena y 4 en el árbol construido por medianas. La implementación no garantiza que se mantenga equilibrado después de nuevas operaciones.

17.4. En hijo izquierdo–hermano derecho, ¿qué conserva la equivalencia estructural y qué interpretación errónea debe evitarse?

Se conservan las unidades, las relaciones originales entre padres e hijos y el orden de los hermanos. El enlace izquierdo apunta al primer hijo y el derecho al siguiente hermano.

Debe evitarse interpretar el enlace derecho como subordinación. Aunque FISEI enlace por la derecha con FCHE, ambas siguen siendo hijas de UTA. Tampoco debe interpretarse esta representación como un ABB.

17.5. ¿Cómo evidencia GitHub que cada integrante implementó y probó una parte real del proyecto?

Mediante ramas de trabajo, commits con cambios sustanciales, diferencias de código, pruebas incorporadas y pull requests revisadas por otro integrante. El README debe relacionar los aportes con esas evidencias.

Los nombres escritos en comentarios o carpetas del ZIP indican una distribución prevista; por sí solos no comprueban la autoría ni que una revisión haya ocurrido en GitHub.

2.2 Referencias bibliográficas

[1] L. E. Vargas Azcona, "Problemas y Algoritmos: capítulos 2–4 (recursión), 8–11 (complejidad), 13 (pilas, colas, listas) y 14 (árboles binarios, recorridos, ABB, complejidad)", Estructura de Datos, Universidad Técnica de Ambato, 2026. Guía de la actividad proporcionada en clase.

[2] EUoss09, "APE3_ArbolGeneral_ABB_Grupo3", código fuente, GitHub, 2026. Disponible: github.com

[3] juanchicoyunda1234, "Arboles", código fuente, GitHub, 2026. Disponible: github.com

[4] Damienq7w, "APE3-Arboles-Grupo2", código fuente, GitHub, 2026. Disponible: github.com

2.3 Anexos

```
== Árbol general: base (Damian) ==
  [OK]    Jerarquía válida: se cargan las 9 unidades y se encuentran por código
  [OK]    Jerarquía inválida: una FACULTAD no puede colgar de una CARRERA
  [OK]    Padre inexistente: se rechaza la unidad
  [OK]    Código duplicado: se rechaza aunque el padre sea válido
  [OK]    Raíz única: no se acepta una segunda raíz ni una raíz que no sea UNIVERSIDAD
  [OK]    Árbol vacío: buscar y agregar no lanzan NullPointerException
```

Figura 1. *Validación de la jerarquía, padre inexistente, códigos duplicados, raíz única y árbol vacío. Seis casos aprobados.*

```
== Árbol general: operaciones (Joel) ==
  [OK]    Aceptación jerarquía inicial: 9 unidades, 5 hojas, altura 3, grado FISEI 2, nivel
    3B 3
  [OK]    Eliminar FISEI con descendencia: rechazo y jerarquía sin cambios
  [OK]    Eliminación segura: la raíz UTA no se elimina; una hoja sí; inexistente no
  [OK]    Transformación hijo-izquierdo / hermano-derecho
  [OK]    Árbol vacío y un solo nodo: altura -1 y 0, sin NullPointerException
```

Figura 2. *Métricas del árbol general, eliminación segura y conversión hijo izquierdo–hermano derecho. Cinco casos aprobados.*

```
== ABB: inserción, búsqueda y recorridos (Mateo) ==
  [OK]    Aceptación: inorden tras 12 inserciones sale ordenado
  [OK]    Preorden y posorden de los datos del enunciado
  [OK]    Aceptación: insertar de nuevo 40 se rechaza y el conteo sigue en 12
  [OK]    Buscar existente y no existente, contando comparaciones
  [OK]    Árbol vacío: recorridos vacíos, buscar null y datos inválidos rechazados
```

Figura 3. *Inserción, búsqueda, recorridos y rechazo de matrículas duplicadas o datos inválidos. Cinco casos aprobados.*

```
== ABB: modificar y eliminar (Luis) ==
  [OK]    Modificar 35: cambia nombre y promedio, la matrícula y la posición se conservan
  [OK]    Modificar con datos inválidos o matrícula inexistente no cambia nada
  [OK]    Inorden antes y después de eliminar 20 (hoja), 60 (un hijo) y 70 (dos hijos)
  [OK]    Eliminar inexistente, eliminar la raíz y vaciar el árbol
```

Figura 4. *Modificación sin alterar la matrícula y eliminación de nodos hoja, con un hijo y condos hijos. Cuatro casos aprobados.*

```
== ABB: métricas y análisis (Sara) ==
  [OK]    Métricas con los 12 estudiantes: altura 3, 6 hojas, mínimo 20, máximo 85
  [OK]    Consistencia del promedio: coincide con el cálculo hecho a mano y se actualiza
  [OK]    Árbol vacío y un nodo: altura -1 / 0, sin NullPointerException
  [OK]    Invariante ABB: válida con los datos y detecta un enlace roto a mano
  [OK]    Análisis: cadena O(n) vs equilibrado O(log n) con el mismo tamaño
```

Figura 5. *Métricas, consistencia del promedio, casos límite, invariante del ABB y análisis de búsqueda. Cinco casos aprobados.*

```
== Integración y auditoría (Josue) ==
  [OK]    Auditoría con los datos del enunciado
  [OK]    Auditoría con árboles vacíos no lanza excepciones
  [OK]    Cambio en vivo: dos matrículas y un nodo académico nuevos (sin respuestas
    codificadas)
```

Figura 6. *Auditoría integrada, árboles vacíos y cambios de datos durante la ejecución. Tres casos aprobados.*

```
----------------------------------------
Pruebas aprobadas: 28 | fallidas: 0 | total: 28
```

Figura 7. *Resumen de la ejecución de EjecutarPruebas: 28 casos aprobados y cero fallos.*

```
1) Jerarquía inicial
UTA - Universidad Técnica de Ambato [UNIVERSIDAD]
  FISEI - Fac. Ing. Sistemas, Electrónica e Industrial [FACULTAD]
    SOFTWARE - Carrera de Software [CARRERA]
      3A - Software Tercero A [PARALELO]
      3B - Software Tercero B [PARALELO]
    TI - Carrera de Tecnologías de la Información [CARRERA]
      4A - TI Cuarto A [PARALELO]
  FCHE - Fac. Ciencias Humanas y de la Educación [FACULTAD]
  FCS - Fac. Ciencias de la Salud [FACULTAD]
   Unidades=9 Hojas=5 Altura=3 Grado FISEI=2 Nivel 3B=3
```

Figura 8*. Jerarquía académica inicial: nueve unidades, cinco hojas, altura tres, grado de FISEI dos y nivel de 3B tres.*

```
2) ABB tras 12 inserciones, inorden: [20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85]
   Preorden: [50, 30, 20, 40, 35, 45, 70, 60, 65, 80, 75, 85]
   Posorden: [20, 35, 45, 40, 30, 65, 60, 75, 85, 80, 70, 50]
3) Insertar 40 otra vez -> rechazado, total = 12
4) Modificar 35 -> 35 - Hugo Sánchez Mod. (9.6)
5) Eliminar 20, 60, 70 -> inorden [30, 35, 40, 45, 50, 65, 75, 80, 85], nodos = 9
6) Eliminar FISEI -> rechazado, unidades = 9
```

Figura 9. *Recorridos iniciales, rechazo de la matrícula duplicada 40, modificación de 35 y eliminación de 20, 60 y 70. Se rechaza eliminar FISEI.*

```
7) Transformación general-binario:
Raíz: UTA
  Izq (primer hijo): FISEI
    Izq (primer hijo): SOFTWARE
      Izq (primer hijo): 3A
        Der (hermano): 3B
      Der (hermano): TI
        Izq (primer hijo): 4A
    Der (hermano): FCHE
      Der (hermano): FCS
```

Figura 10. *Conversión del árbol general: el enlace izquierdo representa al primer hijo y el derecho al siguiente hermano.*

```
8) ========== AUDITORÍA UTA ==========
-- Jerarquía académica (árbol general)
  Total de unidades : 9
  Hojas             : 5
  Altura general    : 3 (aristas)
-- Matrícula (ABB)
  Total estudiantes : 9
  Hojas del ABB     : 4
  Altura del ABB    : 3 (aristas)
  Matrícula menor   : 30 - Bruno Díaz (7.2)
  Matrícula mayor   : 85 - Luis Ortega (8.3)
  Promedio general  : 8.52
  Invariante ABB    : VÁLIDA
==================================
```

Figura 11. *Auditoría posterior al escenario: nueve estudiantes, promedio 8.52, matrículas extremas 30 y 85 e invariante válida.*

```
9) Análisis de búsqueda (n = 12 en ambos ABB)
ABB                   | Altura   | Buscar 85 (existe) | Buscar 90 (no existe)
Cadena (ascendente)   | 11       | 12                 | 12
Equilibrado (medianas) | 3        | 4                  | 4
Búsqueda O(h): en cadena h = n - 1 -> O(n); en el equilibrado h ≈ log2(n) -> O(log n).
```

Figura 12. *Comparación con doce matrículas: las búsquedas de 85 y 90 visitan doce nodos en cadena y cuatro en el árbol construido por medianas.*

El repositorio contiene el código fuente en src, las evidencias y el README con la descripción del problema, la estructuray el .pdf.

Enlace repositorio: https://github.com/Damienq7w/Estructura_Datos_PruebaPractica2.git